Assignment 9: Spatial Analysis in R

Tallulah Bowden

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics (ENV872L) on spatial analysis.

Directions

1. Rename this file `<FirstLast>_A09_SpatialAnalysis.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change "Student Name" on line 3 (above) with your name.
3. Use the lesson as a guide. It contains code that can be modified to complete the assignment.
4. Work through the steps, **creating code and output** that fulfill each instruction.
5. Be sure to **answer the questions** in this assignment document. Space for your answers is provided in this document and is indicated by the ">" character. If you need a second paragraph be sure to start the first line with ">". You should notice that the answer is highlighted in green by RStudio.
6. When you have completed the assignment, **Knit** the text and code into a single **HTML** file.

DATA WRANGLING

Set up your session

1. Import libraries: tidyverse, sf, leaflet, here, and mapview
2. Execute the `here()` command to display the current project directory

```
#1.
library(tidyverse)
library(sf)
library(leaflet)
library(here)
library(mapview)
library(viridis)

#2.
getwd()
```

```
## [1] "/home/guest/Environmental Data Exploration/EDE_Fall2025"
```

```
here()
```

```
## [1] "/home/guest/Environmental Data Exploration/EDE_Fall2025"
```

Read (and filter) county features into an sf dataframe and plot

In this exercise, we will be exploring stream gage height data in Nebraska corresponding to floods occurring there in 2019. First, we will import from the US Counties shapefile we've used in lab lessons, filtering it this time for just Nebraska counties. Nebraska's state FIPS code is `31` (as North Carolina's was `37`).

3. Read the `cb_2018_us_county_20m.shp` shapefile into an sf dataframe, filtering records for Nebraska counties (State FIPS = 31)
4. Reveal the dataset's coordinate reference system
5. Plot the records as a map (using `mapview` or `ggplot`)

```
#3. Read in Counties shapefile into an sf dataframe,
#filtering for just NE counties
counties_sf <- st_read(here("Data/Spatial/cb_2018_us_county_20m.shp")) %>%
  filter(STATEFP == 31)
```

```
## Reading layer 'cb_2018_us_county_20m' from data source
##   '/home/guest/Environmental Data Exploration/EDE_Fall2025/Data/Spatial/cb_2018_us_county_20m.shp'
##   using driver 'ESRI Shapefile'
## Simple feature collection with 3220 features and 9 fields
## Geometry type: MULTIPOLYGON
## Dimension:     XY
## Bounding box:  xmin: -179.1743 ymin: 17.91377 xmax: 179.7739 ymax: 71.35256
## Geodetic CRS:  NAD83
```

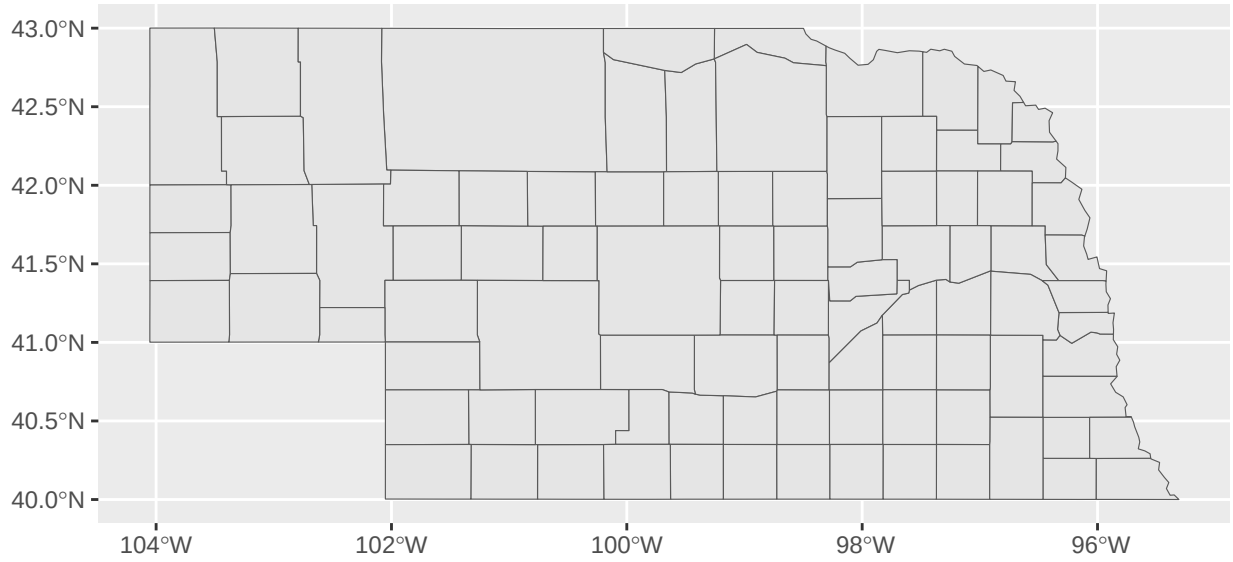
```
#4. Reveal the CRS of the counties features
st_crs(counties_sf)
```

```
## Coordinate Reference System:
##   User input: NAD83
##   wkt:
## GEOGCRS["NAD83",
##     DATUM["North American Datum 1983",
##         ELLIPSOID["GRS 1980",6378137,298.257222101,
##             LENGTHUNIT["metre",1]]],
##     PRIMEM["Greenwich",0,
##         ANGLEUNIT["degree",0.0174532925199433]],
##     CS[ellipsoidal,2],
##         AXIS["latitude",north,
##             ORDER[1],
##             ANGLEUNIT["degree",0.0174532925199433]],
##         AXIS["longitude",east,
##             ORDER[2],
##             ANGLEUNIT["degree",0.0174532925199433]],
##     ID["EPSG",4269]]
```

```
st_crs(counties_sf)$epsg
```

```
## [1] 4269
```

```r
#5. Plot the data
ggplot()+
  geom_sf(data = counties_sf)
```



6. What is the EPSG code of the Counties dataset? Is this a geographic or a projected coordinate reference system? (In other words, does this CRS use angular or planar coordinate units?) To what datum is this CRS associated? (Tip: lookup the EPSG code on https://epsg.io or https://spatialreference.org)

   ANSWER: The EPSG code of the Counties dataset is 4269. This is a geographic coordinate reference system. It uses angular coordinate units and is associated with the North American Datum 1983.

Read in gage locations csv as a dataframe, then display the column names it contains

Next we'll read in some USGS/NWIS gage location data added to the `Data/Raw` folder. These are in the `NWIS_SiteInfo_NE_RAW.csv` file.(See `NWIS_SiteInfo_NE_RAW.README.txt` for more info on this dataset.)

7. Read the `NWIS_SiteInfo_NE_RAW.csv` file into a standard dataframe, being sure to set the `site_no` field as well as other character columns as a factor.

8. Display the structure of this dataset.

```r
#7. Read in gage locations csv as a dataframe
gages  <- read.csv(
  file = here("Data/Raw/NWIS_SiteInfo_NE_RAW.csv"),
  colClasses = c("site_no" = "factor"),
  stringsAsFactors = T)


#8. Display the structure of the dataframe
str(gages)
```

```
## 'data.frame':    175 obs. of  7 variables:
##  $ site_no           : Factor w/ 175 levels "06453600","06454100",..: 1 2 3 4 5 6 7 8 9 10 ...
##  $ station_nm        : Factor w/ 175 levels " BIG SANDY CR AT ALEXANDRIA, NE",..: 117 85 87 70 86 88
##  $ site_tp_cd        : Factor w/ 1 level "ST": 1 1 1 1 1 1 1 1 1 1 ...
##  $ dec_lat_va        : num  42.8 42.4 42.9 42.7 42.8 ...
##  $ dec_long_va       : num  -98.2 -103.8 -100.4 -99.7 -99.3 ...
##  $ coord_acy_cd      : Factor w/ 6 levels "1","5","F","H",..: 6 6 6 6 6 6 6 6 6 6 ...
##  $ dec_coord_datum_cd: Factor w/ 1 level "NAD83": 1 1 1 1 1 1 1 1 1 1 ...
```

9. What columns in the dataset contain the x and y coordinate values, respectively?
   > ANSWER: The column labeled "dec_long_va" contains x coordinate values. The column labeled "dec_lat_va" contains y coordinate values.

Convert the dataframe to a spatial features ("sf") dataframe

10. Convert the dataframe to an sf dataframe. * Note: These data use the same coordinate reference system as the counties dataset
11. Display the structure of the resulting sf dataframe

```r
#10. Convert to an sf object
gages_sf <- st_as_sf(gages, coords = c('dec_long_va','dec_lat_va'),
                         crs=4269)


#11. Display the structure
str(gages_sf)
```

```
## Classes 'sf' and 'data.frame':   175 obs. of  6 variables:
##  $ site_no           : Factor w/ 175 levels "06453600","06454100",..: 1 2 3 4 5 6 7 8 9 10 ...
##  $ station_nm        : Factor w/ 175 levels " BIG SANDY CR AT ALEXANDRIA, NE",..: 117 85 87 70 86 88
##  $ site_tp_cd        : Factor w/ 1 level "ST": 1 1 1 1 1 1 1 1 1 1 ...
##  $ coord_acy_cd      : Factor w/ 6 levels "1","5","F","H",..: 6 6 6 6 6 6 6 6 6 6 ...
##  $ dec_coord_datum_cd: Factor w/ 1 level "NAD83": 1 1 1 1 1 1 1 1 1 1 ...
##  $ geometry          :sfc_POINT of length 175; first list element:  'XY' num  -98.2 42.8
##  - attr(*, "sf_column")= chr "geometry"
##  - attr(*, "agr")= Factor w/ 3 levels "constant","aggregate",..: NA NA NA NA NA
##   ..- attr(*, "names")= chr [1:5] "site_no" "station_nm" "site_tp_cd" "coord_acy_cd" ...
```

12. What new field(s) appear in the sf dataframe created? What field(s), if any, disappeared?

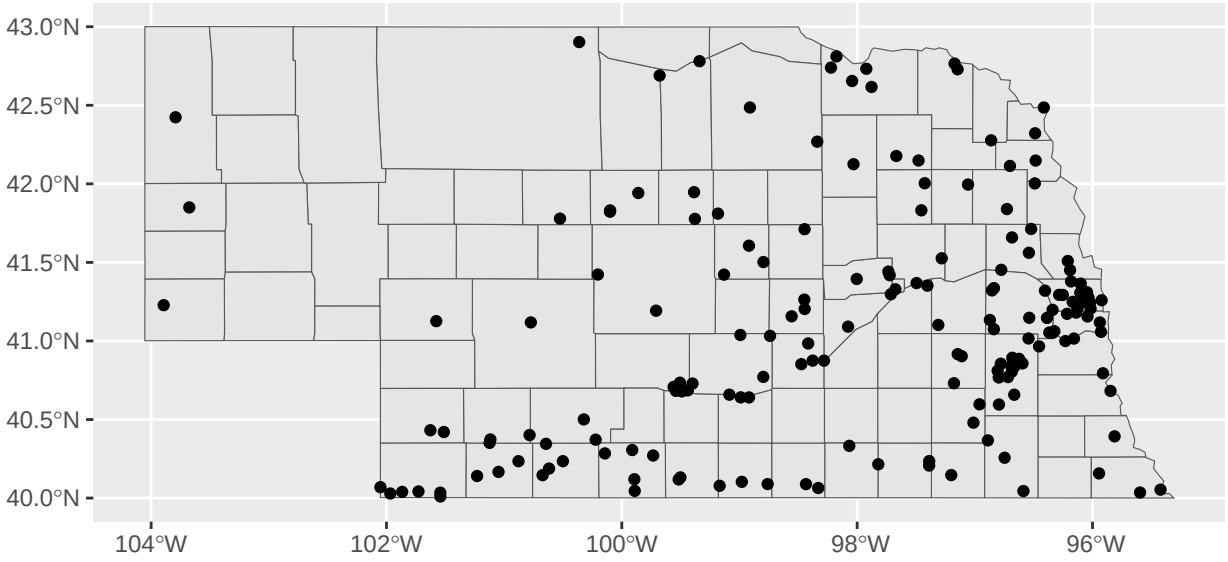
ANSWER: In our new sf dataframe, a new "geometry" column has been created that tells us what type of geometry each row is (point, line, or polygon) and also contains its geographic location as (x, y) coordinates. The columns labeled 'dec_long_va' and 'dec_lat_va' have disappeared since those were used to create the (x, y) coordinates.

Plot the gage locations on top of the counties

13. Use `ggplot` to plot the county and gage location datasets.

- Be sure the datasets are displayed in different colors
- Title your plot "NWIS Gage Locations in Nebraska"
- Subtitle your plot with your name

```
#13. Plot the gage locations atop the county features
ggplot()+
  geom_sf(data = counties_sf)+
  geom_sf(data = gages_sf)
```



Read in the gage height data and join the site location data to it.

Lastly, we want to attach some gage height data to our site locations. I've constructed a csv file listing many of the Nebraska gage sites, by station name and site number along with stream gage heights (in meters) recorded during the recent flood event. This file is titled `NWIS_SiteFlowData_NE_RAW.csv` and is found in the Data/Raw folder.

14. Read the `NWIS_SiteFlowData_NE_RAW.csv` dataset in as a dataframe
    - Pay attention to which fields should be imported as factors!
15. Show the structure of the dataframe.
16. Join our site information (already imported above) to these gage height data
    - The `site_no` and `station_nm` can both/either serve as joining attributes
    - Construct this join so that the result only includes records features where both tables have data (N=136)
17. Show the column names of this resulting spatial dataframe
18. Show the dimensions of the resulting joined dataframe

```
#14. Read the site flow data into a data frame
siteflow <- read.csv(here("Data/Raw/NWIS_SiteFlowData_NE_RAW.csv"),
                         colClasses = c("site_no" = "factor"),
                         stringsAsFactors = T)


#15. Show the structure of the dataframe
str(siteflow)
```

```
## 'data.frame':    152 obs. of  4 variables:
##  $ site_no   : Factor w/ 133 levels "06453600","06453620",..: 1 2 3 3 4 5 6 6 7 8 ...
##  $ station_nm: Factor w/ 133 levels "Antelope Creek at 27th Street at Lincoln, Nebr.",..: 89 61 65 65
##  $ date      : Factor w/ 20 levels "2018-12-11 01:00:00",..: 13 13 13 13 3 19 4 4 8 5 ...
##  $ gage_ht   : num  12.58 19.96 3.61 3.63 11.38 ...
```

```
#16. Join the flow data to our NWIS gage location spatial dataframe
siteflow_gages_sf_join <-  merge(
  x = gages_sf,
  y = siteflow,
  by.x = "site_no",
  by.y = "site_no"
  )%>%
  select(site_no, station_nm.x, site_tp_cd, coord_acy_cd, dec_coord_datum_cd,
         date, gage_ht, geometry)


#17. Show the column names in the resulting spatial dataframe
colnames(siteflow_gages_sf_join)
```

```
## [1] "site_no"            "station_nm.x"       "site_tp_cd"
## [4] "coord_acy_cd"       "dec_coord_datum_cd" "date"
## [7] "gage_ht"            "geometry"
```

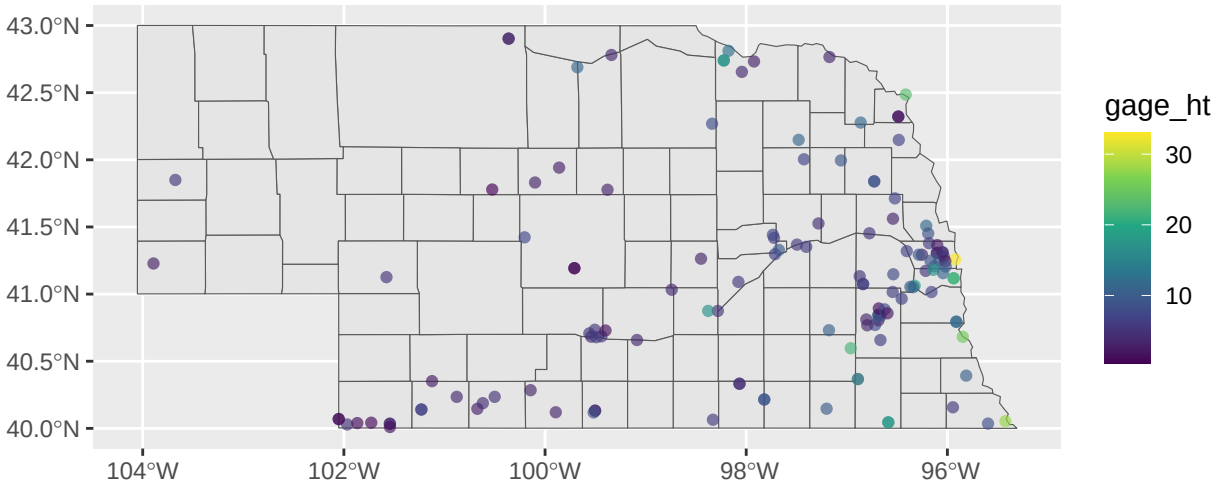
```
#18. Show the dimensions of this joined dataset
dim(siteflow_gages_sf_join)
```

```
## [1] 136   8
```

Map the pattern of gage height data

Now we can examine where the flooding appears most acute by visualizing gage heights spatially. 19. Plot the gage sites on top of counties (using `mapview`, `ggplot`, or `leaflet`) * Show the magnitude of gage height by color, shape, other visualization technique.

```
#Map the points, sized by gage height
ggplot()+
  geom_sf(data = counties_sf)+
  geom_sf(data=siteflow_gages_sf_join, aes(color = gage_ht), alpha = 0.65)+
  scale_color_viridis_c()
```



SPATIAL ANALYSIS

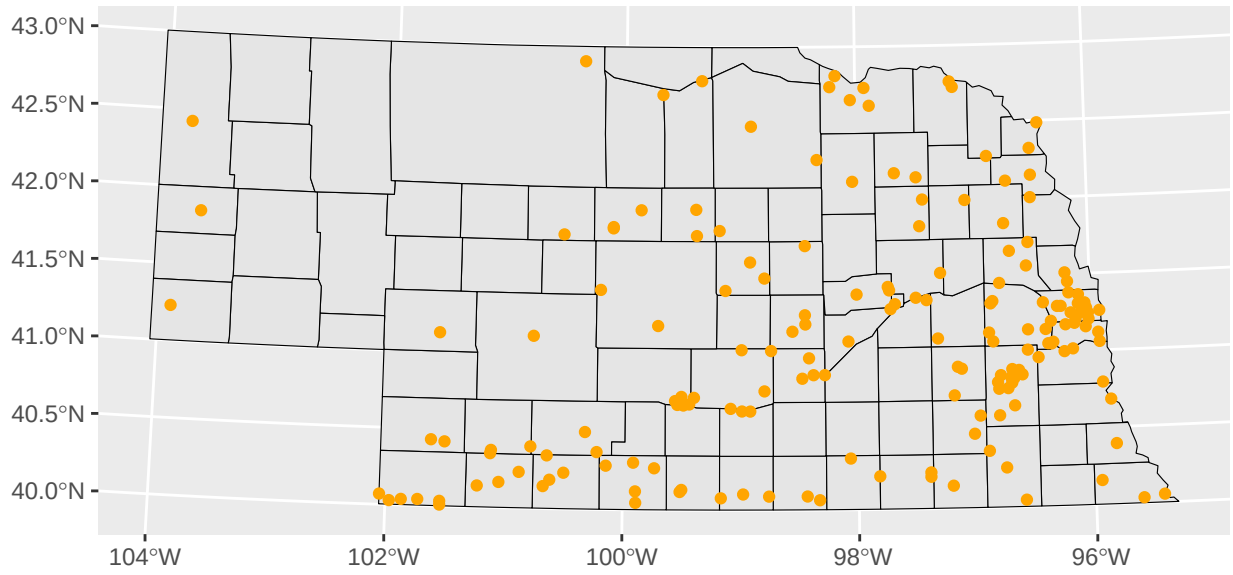
Up next we will do some spatial analysis with our data. To prepare for this, we should transform our data into a projected coordinate system. We'll choose UTM Zone 14N (EPGS = 32614).

Transform the counties and gage site datasets to UTM Zone 14N

20. Transform the counties and gage sf datasets to UTM Zone 14N (EPGS = 32614).
21. Using `ggplot`, plot the data so that each can be seen as different colors.

```r
#20 Transform the counties and gage location datasets to UTM Zone 14
counties_sf_utm <- st_transform(counties_sf, crs = 32614)
gages_sf_utm <- st_transform(gages_sf, crs = 32614)
siteflow_gages_sf_utm <- st_transform(siteflow_gages_sf_join, crs = 32614)


#21 Plot the data
ggplot()+
  geom_sf(data = counties_sf_utm, color = "black")+
  geom_sf(data= gages_sf_utm, color = "orange")
```



Select the gages falling within a given county

Now let's zoom into a particular county and examine the gages located there. 22. Select Washington county from your projected county sf dataframe 23. Select the gage sites falling within that county to a new spatial dataframe 24. Select the gage sites within 20km of the county to a new spatial dataframe 25. Create a plot showing (each symbolized distinctly): * all Nebraska counties, * the selected county, * the gage sites in that county, * and the gage sites within 20 km of the county

```r
#22 Select Washington county to a new spatial dataframe
washington_county <- filter(counties_sf_utm, NAME == "Washington")

#23 Spatially select gages within the selected county to a new spatial dataframe
washington_intersect <- gages_sf_utm %>%
```

```
  st_filter(washington_county, .predicate = st_intersects)

#24 Spatially select gages within 20k of the selected county to a
#new spatial dataframe
buffer_washington <- washington_county %>%
  st_buffer(.,dist=20000)
resultMask <- st_intersects(gages_sf_utm,
                            buffer_washington,
                            sparse = F)
selGages <- gages_sf_utm[resultMask,]

#25 Plot
ggplot()+
  geom_sf(data = counties_sf_utm, color = "grey", fill = "white")+
  geom_sf(data = washington_county, color = "black", fill = "yellow")+
  geom_sf(data = selGages, color = "black", alpha = 0.65)+
  geom_sf(data = washington_intersect, color = "red")
```